

Java Lambdas, Method References & Collectors — Cheat Sheet

Quick-recall reference built from your Q&A session

1. Lambda vs Method Reference — The Core Rule

Lambda = write your own code, any number of steps.

Method reference = point at ONE existing method, no extra steps, no extra logic.

A method reference can replace a lambda **only** when the lambda body is exactly one single method call whose input/output types already match what's expected. The moment you need two actions chained, a comparison, or any extra logic — you need a lambda.

Examples across functional interface types

```
// Runnable – zero args, void return
Runnable r1 = () -> System.out.println("Hello");
Runnable r2 = YourClass::greet; // if greet() is static void

// Consumer<T> – one arg, void return
Consumer<String> c1 = s -> System.out.println(s);
Consumer<String> c2 = System.out::println; // same thing, shorter

// Function<T,R> – one arg, returns a value
Function<Employee, String> f1 = e -> e.getName();
Function<Employee, String> f2 = Employee::getName;

// BiFunction / Comparator – two args
Comparator<Integer> comp1 = (a, b) -> Integer.compare(a, b);
Comparator<Integer> comp2 = Integer::compare;
```

Where method reference FAILS (two actions chained)

```
// Works – lambda, custom code, two steps allowed
employees.forEach(e -> System.out.println(e.getName()));

// FAILS – method ref can only do ONE action, wrong return type too
employees.forEach(Employee::getName); // getName() returns String, forEach needs void

// FIX – split into two single-action stages
employees.stream()
    .map(Employee::getName) // action 1: Employee -> String
    .forEach(System.out::println); // action 2: String -> print
```

Common mistakes recap

```
// WRONG – comparing a method reference to a number (not a call!)
employees.stream().filter(Employee::getSalary > 30);
// FIX – need extra logic (the > comparison) => lambda required
employees.stream().filter(e -> e.getSalary() > 30);

// WRONG – bare method reference as lambda body, emp never used, nothing executes
e.getValue().forEach(emp -> System.out::print);
// FIX option 1 – actually call it
e.getValue().forEach(emp -> System.out.print(emp));
// FIX option 2 – proper method reference, no lambda wrapper needed
e.getValue().forEach(System.out::print);
```

2. Collectors.groupingBy — The 3 Arguments

`Collectors.groupingBy(classifier, mapFactory, downstream)`

Argument	Question it answers	Type expected	Example
----------	---------------------	---------------	---------

classifier	What's the key?	Function<T, K>	Employee::getDepartment
mapFactory	What box holds the WHOLE result? (zero-arg!)	Supplier<Map<K,V>>	TreeMap::new HashMap::new
downstream	What happens to EACH group's values?	Collector<T,?,D>	Collectors.toSet() Collectors.counting()

Critical mental model — they run in PARALLEL, not in sequence

mapFactory and **downstream** never talk to each other. Both independently answer to the classifier's grouping. **mapFactory** decides the shape of the WHOLE result (map type + key order). **downstream** decides the shape of EACH value (what happens inside one bucket of grouped items). Java combines both answers into the final Map at the end.

employees

```
-> classifier splits into buckets: "Eng" -> [e1,e2,e3], "Sales" -> [e4,e5]
-> downstream transforms EACH bucket independently: toSet() -> {e1,e2,e3}, {e4,e5}
-> mapFactory says which box holds it all: TreeMap<String, Set<Employee>>
```

Full worked example

```
employees.stream().collect(Collectors.groupingBy(
    Employee::getDepartment,           // 1. KEY = department name
    TreeMap::new,                      // 2. Map type = TreeMap (sorted keys)
    Collectors.collectingAndThen(      // 3. VALUE =
        Collectors.toList(),           // collect group into a List...
        list -> {                      // ...then sort that list by age
            list.sort(Comparator.comparing(Employee::getAge));
            return list;
        }
    )
));
```

Common groupingBy bugs

```
// BUG: mapFactory must build a Map, never a List/Set
groupingBy(Employee::getDepartment, ArrayList::new, Collectors.counting()); // FAILS

// BUG: counting() returns Long, not Integer -- match your variable's type!
Map<String, Integer> m = ...groupingBy(..., Collectors.counting()); // FAILS
Map<String, Long> m = ...groupingBy(..., Collectors.counting()); // FIX

// BUG: TreeMap's comparator must match the KEY type, not the original object type
new TreeMap<Employee>(...) // wrong if keys are actually department Strings
new TreeMap<String>(Comparator.naturalOrder()) // correct
```

3. collectingAndThen — Post-process a Collector's Result

Collectors.collectingAndThen(collector, finisher)

Step 1 (collector): does the real collecting work, e.g. toList() builds the List.

Step 2 (finisher): a Function that takes Step 1's result, lets you modify/process it, and must return the final value. Analogy: toList() packs the box; the finisher opens it, rearranges the items, reseals it.

```
Collectors.collectingAndThen(
    Collectors.toList(),               // STEP 1: build the List
    list -> { list.sort(...); return list; } // STEP 2: sort it, then return it
)
```

// Rounding a single average to 2 decimals -- same trick, works on ANY result type

```
Collectors.collectingAndThen(
    Collectors.averagingDouble(Employee::getSalary), // STEP 1: compute avg (Double)
    avg -> Math.round(avg * 100.0) / 100.0           // STEP 2: round to 2 decimals
)
```

```
)
// Math.round(avg * 100.0) / 100.0 -> shift right 2 decimals, round, shift back
```

4. max() / min() Trap + mapping()

*Stream.max() / Stream.min() return the **WHOLE** object wrapped in Optional<T> — not the field you compared on!*

```
// WRONG expectation: "I compared by salary so I get a Double back" -- NO.
c.stream().max(Comparator.comparing(Employee::getSalary)); // returns Optional<Employee>

// If you want the Employee object:
Collectors.maxBy(Comparator.comparing(Employee::getSalary)) // -> Optional<Employee>
// no need to manually toSet() then re-stream -- maxBy/minBy do it directly

// If you want JUST the salary (a Double), extract first with mapping():
Collectors.mapping(
    Employee::getSalary, // pull out just the salary
    Collectors.maxBy(Comparator.naturalOrder()) // then max on the Doubles -> Optional<Double>
)

// Unwrap Optional -> plain Double using collectingAndThen
Collectors.collectingAndThen(
    Collectors.mapping(Employee::getSalary, Collectors.maxBy(Comparator.naturalOrder())),
    Optional::get // or .orElse(0.0) to be safer against empty groups
)
```

5. Full Collectors Catalogue (by category)

Category	Methods	Returns
Basic gather	toList(), toSet(), toUnmodifiableList(), toCollection(TreeSet::new)	List / Set
Grouping	groupingBy(classifier) groupingBy(classifier, downstream) groupingBy(classifier, mapFactory, downstream) partitioningBy(predicate)	Map<K,V> (partitioningBy: Map<Boolean,V>)
Joining strings	joining() joining(", ") joining(", ", "[", "]")	String
Math / aggregation	counting() summingInt/Double() averagingInt/Double() summarizingDouble() maxBy(cmp) / minBy(cmp)	Long / Integer / Double SummaryStatistics Optional<T>
Transform then collect	mapping(mapper, downstream) filtering(predicate, downstream) flatMap(mapper, downstream)	depends on downstream
Post-process result	collectingAndThen(downstream, finisher)	whatever finisher returns
Direct map building (one entry per element)	toMap(keyMapper, valueMapper) toMap(key, value, mergeFn) toMap(key, value, mergeFn, mapFactory)	Map<K,V>
Parallel combine (Java 12+)	parallelJoining(collector1, collector2, merger)	merger's return type

*90% of real usage = `toList`, `toSet`, `groupingBy`, `partitioningBy`, `joining`, `counting`, `mapping`, `collectingAndThen`.
Don't memorize the rest — just remember the categories exist, and look up the exact method when you need it.*